

ML Experiment Triage

Ranked, significance tested comparison of machine learning training runs

Olajide Badejo

5 August 2026

Abstract

Comparing training runs by looking at curves fails in four compounding ways: the final point is mostly noise, curve values are autocorrelated enough to break the usual tests, run to run variance is routinely larger than the effects being claimed, and several metrics are compared at once. This report describes a tool that replaces that practice with a stated hypothesis, a permutation test whose assumptions the data actually satisfies, a two gate regression rule with false discovery control, and, most importantly, a calibration suite that measures the tool's own error rates rather than assuming them.

The measured type I error is 4.53 percent against a nominal 5, with power of 96.25 percent on a large effect. The report also publishes what the weaker single run mode costs: on synthetic runs with realistic seed variance and a true effect of exactly zero, comparing one run against one run calls the difference significant on up to 88 percent of cases, where the seed replicated mode holds at 4.1 percent. That number, rather than any feature, is the argument for the design.

Contents

1	Introduction	2
1.1	Objectives	2
1.2	What this report contains	2
2	Background	3
2.1	Autocorrelation	3
2.2	Multiple comparisons	3
2.3	Seed variance	3
3	Methodology	4
3.1	The statistic	4
3.2	Mode one: seed replicated	4
3.3	Mode two: single run window block	4
3.4	Two gates	5
3.5	Sensitivity	5
4	Implementation	6
4.1	Model and storage	6
4.2	Demo sweep	6
5	Results	7
5.1	Measured calibration	7
5.1.1	The designs the test is not exact in	8
5.1.2	What calibration testing changed	8
5.2	The cost of the weaker mode	9
5.3	Demo verdicts	9
5.4	Sensitivity	11
6	Discussion	12
6.1	What this tool is honest about	12
6.2	What I would do differently	12
6.3	Future work	12
7	Conclusion	13
A	Reproducing this report	14

Chapter 1

Introduction

The routine this project replaces is familiar. Two runs, a TensorBoard tab, one loss curve slightly below the other at the right hand edge, and a decision made on it. It is quick, it feels evidential, and it is close to worthless.

The aim was to build something that answers a sharper question: which runs beat the baseline, by how much, with what statistical confidence, and, crucially, with the tool's own calibration proven by tests rather than asserted in a README. The last clause is the one that shaped everything else. A tool that produces p values is only useful if its p values mean what they say, and the only way to know that is to run it thousands of times against data whose truth you set yourself and count how often it is wrong.

1.1 Objectives

1. Parse TensorBoard event files, CSV and JSONL logs into one experiment model, stored in SQLite so nothing is ever reparsed.
2. Compare any run against a baseline with a permutation test on a final window statistic, reporting effect size, confidence interval and p value, in two clearly labelled modes.
3. Flag regressions on two gates, statistical and practical, with Benjamini Hochberg correction across metrics and ranking by severity.
4. Rank hyperparameter sensitivity, always reporting the sample size behind each correlation.
5. Treat statistical calibration as a first class test.
6. Produce a self contained HTML report and reproducible PDFs.

1.2 What this report contains

Chapter 2 sets out why curve comparison needs care. Chapter 3 defines the statistic and both modes precisely, and states what each can and cannot claim. Chapter 4 covers the engineering. Chapter 5 gives the measured calibration, the demo verdict table and the sensitivity analysis. Chapter 6 discusses the limitations honestly, and Chapter 7 concludes.

Chapter 2

Background

2.1 Autocorrelation

Consecutive evaluations of a training run are correlated: the model at step 5000 is nearly the model at step 4999. A test that treats a window of curve points as independent observations computes a standard error far too small and reports significance that is not there. This is not a small inefficiency. It is the difference between a test that controls its error rate and one that does not.

This is why the permutation test here uses the *run* as its unit of analysis, and why the single run mode has to permute blocks rather than points.

2.2 Multiple comparisons

Testing eight metrics at $\alpha = 0.05$ gives roughly a one in three chance of at least one false positive when nothing has changed. Any tool that compares several metrics at once and does not correct for it is manufacturing findings.

Benjamini and Hochberg's step up procedure controls the false discovery rate and is the right instrument here, because training metrics are strongly correlated and Bonferroni assumes worst case dependence. On the fifteen p values in the original 1995 paper, Bonferroni rejects three where the step up procedure rejects four; that difference in power is real, and this project's implementation is tested against exactly that example.

2.3 Seed variance

Two identical configurations trained with different seeds land in different places, and in deep learning that spread is routinely larger than the effect sizes claimed from single runs. Bouthillier and colleagues set this out in detail in their MLSys 2021 accounting of variance in benchmarks, and it is the finding this tool is built around.

The consequence is sharper than "be careful". A comparison with one run per side contains *no information* about run to run variance. It cannot separate the effect of a change from the effect of a seed, not imprecisely but in principle. Section 5.2 measures what that costs.

Chapter 3

Methodology

3.1 The statistic

For a tagged series of n values, smooth with a centred moving average of width nine, then take the mean of the final window: the last $k = \max(20, \lceil 0.1n \rceil)$ points. The full specification as used is:

final window mean over the last $\max(20, 10\% \text{ of steps})$ points of a 9 point moving average;
two sided permutation test at alpha 0.05; seed 20260805

The window mean has roughly $\sqrt{k/\tau}$ times less noise than a single final point, where τ is the integrated autocorrelation time. Using the whole curve instead would mix how fast a run got going with how good it got, answering neither question.

3.2 Mode one: seed replicated

Null hypothesis. The condition label is exchangeable across runs; the final window statistics of the two conditions are draws from one distribution.

Test. Pool the $m + n$ run level statistics, and over every split into groups of size m and n compute the difference of group means. The p value is the fraction at least as extreme in absolute value as the observed difference. With five seeds a side there are $\binom{10}{5} = 252$ arrangements, enumerated exhaustively, so the p value is exact rather than estimated.

Why this is the strong claim. Seed variance sits inside the null distribution rather than being assumed away. If two conditions differ by less than the runs of one condition differ among themselves, the permutation distribution is wide and the p value is large, which is correct.

The design limit, reported not hidden. With three seeds a side there are twenty distinct assignments, so the smallest attainable two sided p value is 0.1 and no result can clear $\alpha = 0.05$. The tool computes this bound, warns when it exceeds alpha, and classifies such findings as inconclusive rather than as no change. A verdict of no significant difference from a design that could not have found one is not a finding.

3.3 Mode two: single run window block

Null hypothesis. The two final windows are blockwise exchangeable: both are segments of the same stationary process.

Test. Split each final window into contiguous blocks of 4 times the estimated integrated autocorrelation time, estimated on each window separately, and permute the block labels between the runs.

Precondition, enforced. The window must hold at least eight such blocks. Where it does not, the tool raises an error naming the remedy rather than shortening the block to fit. Section 5.1 explains why that distinction is not fastidiousness.

What it cannot claim. Its null is that these two windows come from the same process, which is not the question anyone is really asking. Every output from this mode is labelled a weaker claim.

3.4 Two gates

A finding is a regression only when both hold:

two gates: Benjamini Hochberg at FDR 5% (adjusted p at or below 0.05, configurable with `--fdr`), and a relative effect of at least 2 percent; alpha 0.05 bounds admissibility only

Direction is inferred from the tag name and can be overridden. Findings are ranked by severity, which scales the size of the harm by confidence and halves the result for the weaker mode, so a weaker claim never outranks a stronger claim of the same size.

3.5 Sensitivity

Spearman rank correlation between each numeric hyperparameter and the final window statistic, with the run count printed beside every correlation. Rank correlation rather than ANOVA, because sweeps are unbalanced and hyperparameter effects are usually monotone but strongly nonlinear.

Chapter 4

Implementation

4.1 Model and storage

Every parser produces one **Experiment**: an identifier, a configuration, and a set of tagged step series. Everything downstream reads that model and nothing else, which is what lets a TensorBoard run and a JSONL log be compared without the analysis knowing where the numbers came from. The unit suite asserts this directly: four committed fixtures encoding one identical series in four containers parse to numerically identical experiments.

Series are held and stored as float32. Holding float64 in memory while writing float32 would make a store round trip lossy in a way no test could honestly call exact; fixing the model at the storage precision makes the round trip a bit for bit assertion.

Ingest fingerprints each source on path, size and modification time. An unchanged source is skipped, a changed one replaces its rows in a single transaction, and an interrupted ingest loses only the run in flight. Content hashing was rejected because hashing a directory of event files costs about as much as parsing it.

4.2 Demo sweep

The demo generates 31 runs across 7 conditions, in three log formats at once, from fixed seeds. Table 4.1 lists them.

Condition	Log format	Seeds	Final loss	Spread
lr0.0003_bs32	tensorboard	5	0.4135	0.0124
lr0.0010_bs32 (baseline)	tensorboard	5	0.3438	0.0158
lr0.0010_bs64	jsonl	5	0.3441	0.0253
lr0.0030_bs128	csv	1	0.2909	0.0000
lr0.0030_bs32	csv	5	0.2767	0.0241
lr0.0030_bs64	jsonl	5	0.3005	0.0255
lr0.0100_bs32	csv	5	0.4768	0.0159

Table 4.1: The synthetic demo sweep. Spread is the standard deviation across seeds within a condition, and it is the quantity every claim in the next chapter has to overcome.

The database holds 372000 points across 62 series in 2036 KB, a compression ratio of 2.5 on the series blobs.

Chapter 5

Results

5.1 Measured calibration

This is the section the rest of the report depends on.

Measurement	Result	Gate
Type I error, seed replicated	4.53 percent (+/- 0.74), 3000 null cases	[2, 8] percent
Type I error across seed variance	4.40, 4.70, 5.00 percent, sigma 0.005 to 0.05	[2, 8] percent
Power, seed replicated	96.25 percent, 800 cases	above 90 percent
Type I error, window block	6.28 percent (+/- 1.07), 1989 cases, 11 refused	[2, 8] percent
Power, window block	100 percent, 300 cases	above 90 percent
Type I error, paired clustered	4.90 percent (+/- 1.34), 1000 cases, 10 clusters of 4 pairs	[2, 8] percent
Null p value uniformity	0.0580, 0.0993, 0.2407, 0.4847, 1500 cases	within 3 standard errors

Table 5.1: Measured error rates against the phase gates. Every case is driven from a fixed seed, so these numbers are deterministic and reproduced by `make test-stats`, 23,900 comparisons in 110 s.

Checking the error rate at a single threshold can be passed by a test that is wrong in two compensating directions, so the suite also checks that the null distribution of p values is uniform.

Nominal threshold	Measured rejection rate
0.05	0.0580
0.10	0.0993
0.25	0.2407
0.50	0.4847

Table 5.2: Rejection rates under the null at four thresholds. A calibrated test is right everywhere, not only at 0.05.

5.1.1 The designs the test is not exact in

Those gates measure the design a permutation test is exact in: the same number of runs on both sides, drawn with the same spread. A real sweep is neither. The setting that was already trusted has usually been run the most times and moves the least, and it is that asymmetry, rather than the sample size, that breaks the guarantee. Every cell of Table 5.3 has a true effect of exactly zero, so every rejection in it is a false positive.

Design	Type I error	Gate
Unequal spread, 5 narrow against 5 wide	7.83 percent (+/- 1.52), 1200 null cases, sigma 0.01 against 0.05	[2, 10] percent
Unequal spread, 3 narrow against 7 wide	2.08 percent (+/- 0.81), 1200 null cases, sigma 0.01 against 0.05	[1, 15] percent
Unequal spread, 7 narrow against 3 wide	12.92 percent (+/- 1.90), 1200 null cases, sigma 0.01 against 0.05	[1, 15] percent
Unequal counts only, 7 against 3	4.25 percent (+/- 1.14), 1200 null cases, one spread	[2, 8] percent
Heavy tailed noise, Student t at 3 df	5.00 percent (+/- 1.23), 1200 null cases, 5 runs a side	[2, 8] percent
Paired, clustered, macro F1	2.00 percent (+/- 1.23), 500 null cases, a discrete statistic	at most 8 percent
Paired, clustering ignored	12.10 percent, 1000 null cases, the same clustered data	measured, not gated

Table 5.3: Type I error at a nominal 5 on designs that break plain exchangeability. The quantity permuted is the Welch t , which is what holds the first five rows near nominal; the sixth is conservative because macro F1 over forty units is discrete; the seventh is what ignoring a known clustering costs.

The three gates differ because the three regimes do. Equal spreads and heavy tails are exact and keep the ordinary [2, 8] percent. Unequal spreads with five or more runs a side get [2, 10]: studentizing is asymptotic in the number of runs, so five a side is close to nominal rather than on it. Unequal spreads with fewer than five runs on one side get [1, 15], set to catch the defect rather than to assert an exactness the design cannot deliver: permuting the raw mean difference measured 17.92 percent in that cell, and permuting the studentized statistic measures 12.92 percent. The tool warns on exactly that design.

5.1.2 What calibration testing changed

The first full run of the suite measured the window block mode at a type I error of **12 percent** against a nominal 5. Two causes, both found by measurement rather than by reading the code.

First, the autocorrelation time was estimated on the two windows concatenated. Joining two independent runs end to end puts a step change at the join, which the estimator reads as long range dependence, inflating the estimate by roughly a factor of two.

Second, and more consequentially, the block length was one autocorrelation time. At one τ the block means are still visibly correlated with their neighbours. The observed split is contiguous, one run's blocks against the other's, but a permuted split scatters neighbouring blocks across both groups, breaking that correlation and producing a null distribution narrower than the truth. A null that is too narrow is exactly how a test becomes anticonservative.

The fix was to estimate τ per window, on the raw values rather than on the smoothed ones, and to set the block at a multiple of it. That multiple was swept again once the estimator was corrected, because a multiplier chosen against an inflated τ is not derived from anything: measured over 2000 null cases a point, type I error runs 8.85, 7.02, 6.28, 6.09 and 6.03 percent at two, three, four, five and six τ , against refusal rates on 4000 step runs of 0, 6, 20, 40 and 78 percent. The rate is asymptotic at about 6 percent, so four τ buys three quarters of the available correction and every longer block buys almost nothing while refusing far more. That created a third decision: at 4 τ a short run may not contain eight blocks, and shortening the block to fit is precisely the failure above. So the mode refuses. A quietly miscalibrated p value is worse than no p value, because it spends credibility that nothing later can recover.

5.2 The cost of the weaker mode

Table 5.4 is the most useful number this project produced. Both modes are run on synthetic runs carrying realistic seed variance, with a true effect of *exactly zero*. Every rejection is a false positive.

Seed standard deviation	Window block mode	Seed replicated mode
0.01	54.3 percent	4.1 percent
0.02	76.6 percent	3.8 percent
0.04	87.7 percent	4.1 percent

Table 5.4: False positive rates when the true effect is zero and seed variance is present. Comparing one run against one run, on data where nothing is different, calls it a significant difference most of the time.

This is not a defect in the implementation. It is what the question “are these two curves different” can answer when it has no information about how much a curve moves between seeds. The tool still offers the mode, because sometimes one run is all there is, and labels every result from it accordingly.

5.3 Demo verdicts

The tool identifies lr0.0030_bs32 as the best condition, which is the ground truth the sweep was built with, and does not flag the condition whose true effect is real but below the practical threshold.

One row deserves a second look. The single seed condition reports an adjusted p of 0.0006, far smaller than the five seed conditions with comparable effects, which sit between 0.016 and 0.032. That is the anticonservatism of Table 5.4 appearing in the demo itself rather than only in the calibration suite.

Candidate	Metric	Change	p	p adj.	Verdict
lr0.0100_bs32	val/loss	+38.67%	0.0079	0.0198	regression
lr0.0003_bs32	val/loss	+20.27%	0.0079	0.0198	regression
lr0.0100_bs32	val/accuracy	-5.03%	0.0079	0.0198	regression
lr0.0030_bs32	val/loss	-19.52%	0.0079	0.0198	improvement
lr0.0030_bs128_seed0 †	val/loss	-16.65%	< 0.0001	< 0.0001	improvement
lr0.0030_bs64	val/loss	-12.59%	0.0238	0.0340	improvement
lr0.0030_bs128_seed0 †	val/accuracy	+1.13%	< 0.0001	< 0.0001	significant but below the practical threshold
lr0.0003_bs32	val/accuracy	-1.99%	0.0238	0.0340	significant but below the practical threshold
lr0.0030_bs32	val/accuracy	+0.96%	0.0238	0.0340	significant but below the practical threshold
lr0.0030_bs64	val/accuracy	+0.91%	0.0714	0.0893	no significant change
lr0.0010_bs64	val/accuracy	-0.38%	0.4762	0.5291	no significant change
lr0.0010_bs64	val/loss	+0.08%	0.9762	0.9762	no significant change

Table 5.5: All 12 comparisons against lr0.0010_bs32, ranked by severity: 3 regressions and 3 improvements. Rows marked † come from the weaker single run mode. Permutation seed 20260805.

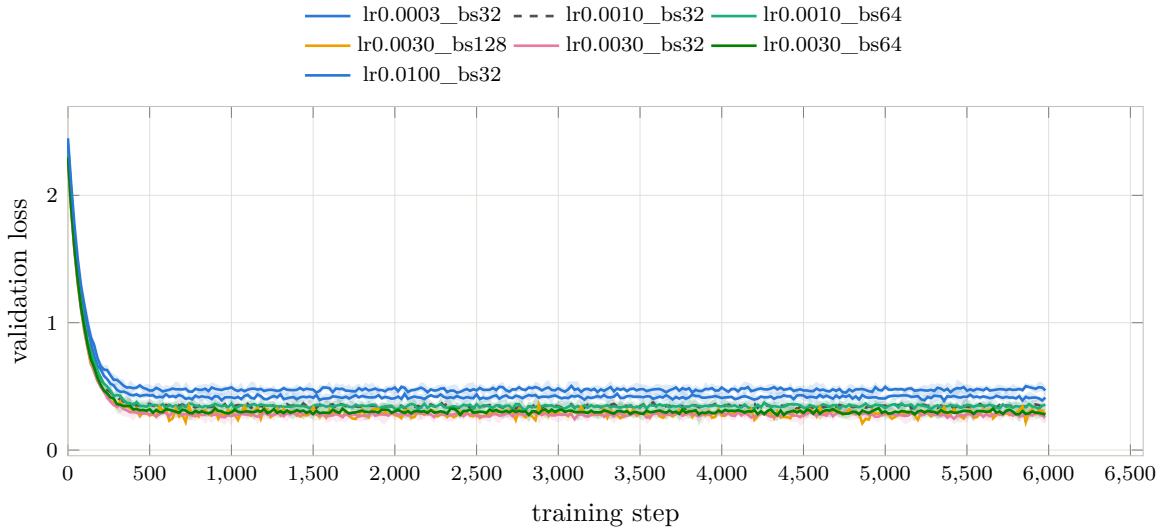


Figure 5.1: Validation loss. Each line is the mean across seeds of a smoothed curve; the band is the full range across seeds of that condition. Where two bands overlap, the conditions are not separated by more than their own run to run noise, whatever the lines appear to do.

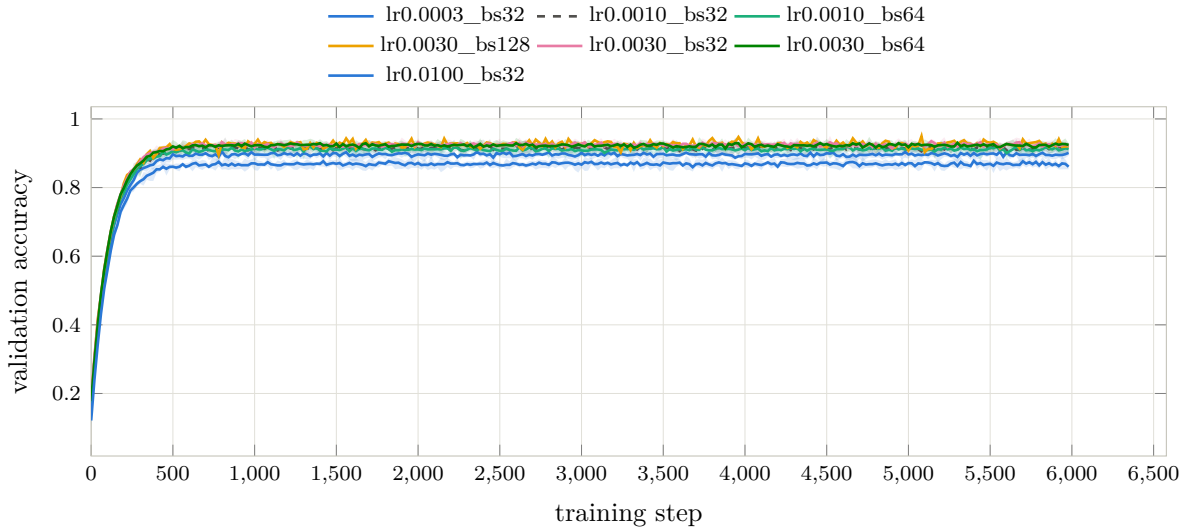


Figure 5.2: Validation accuracy for the same runs.

5.4 Sensitivity

Parameter	Metric	Spearman ρ	p	p adj	n variants	n runs	Distinct values
batch_size	val/accuracy	+0.538	0.2286	0.7381	7	31	3
batch_size	val/loss	-0.359	0.4571	0.7381	7	31	3
learning_rate	val/accuracy	+0.168	0.7381	0.7381	7	31	4
learning_rate	val/loss	-0.168	0.7381	0.7381	7	31	4

Table 5.6: Spearman rank correlation of each swept hyperparameter against each metric, over all 31 runs.

Table 5.6 demonstrates a real limitation rather than hiding it. Learning rate drives by far the largest effects in this sweep, and shows the *weaker* correlation. The reason is that the sweep gives learning rate an optimum in the middle of its range: 0.0003 is worse than 0.001, 0.003 is the best of the four and 0.01 is by a distance the worst. That is a U shape, and the rank correlation of a U shape is near zero however large the effect. Batch size, swept monotonically over the same runs, correlates more strongly while driving a smaller effect.

A near zero rank correlation here means *not monotone*, never *no effect*. The tool states this wherever the table appears.

Chapter 6

Discussion

6.1 What this tool is honest about

Three limitations are documented, tested and printed rather than smoothed over: the single run mode cannot see seed variance and says so; rank correlation is blind to non monotone effects and the demo demonstrates it; and sensitivity is association across a sweep, not a controlled effect.

Documenting a limitation is cheap. Building the demonstration of it into the demo, and pinning it with a test named for it so nobody later “fixes” the inconvenient number, is the part that took effort and is the part worth having.

6.2 What I would do differently

The Welch interval reported alongside the seed replicated p value leans on a normal approximation the p value itself does not need. It is named in the output for that reason. A percentile bootstrap over five seeds is known to be poor, so the alternative was not obviously better, but a properly studentised bootstrap would be more consistent.

The severity ranking is a defensible heuristic rather than a derived quantity. Halving the weaker mode encodes a judgement about how much less a weak claim is worth, and that number is not measured.

6.3 Future work

A nonlinear dependence measure alongside Spearman, such as distance correlation, would see the U shape that rank correlation misses. A sequential mode that decides how many further seeds are needed to resolve an inconclusive comparison would be more useful than any of the above, since the most common honest verdict this tool gives is that there is not enough data yet.

Chapter 7

Conclusion

The tool does what it set out to do: it takes a directory of runs in three log formats, and returns a ranked table saying which beat the baseline, by how much, with what confidence, and under which of two clearly separated claims.

What makes it worth trusting is not any of that. It is that its error rates were measured rather than assumed, that the measurement found a real defect running at better than twice the nominal error rate, and that the fix was to make the tool refuse rather than to widen the gate. The calibration suite is the product; the comparison is what it certifies.

Appendix A

Reproducing this report

```
make env      # Python 3.13 venv, pinned dependencies
make test     # full suite including the calibration gates
make demo     # regenerate the sweep, database and HTML report
make all      # everything, including this PDF
```

Every table and figure above was generated by `scripts/gen_report_assets.py` from the database, at permutation seed 20260805. No number in this document was typed by hand.

Bibliography

- [1] Y. Benjamini and Y. Hochberg. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, 57(1):289 to 300, 1995.
- [2] X. Bouthillier et al. Accounting for variance in machine learning benchmarks. *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- [3] P. Good. *Permutation, Parametric, and Bootstrap Tests of Hypotheses*, 3rd edition. Springer, 2005.
- [4] B. Phipson and G. K. Smyth. Permutation p values should never be zero. *Statistical Applications in Genetics and Molecular Biology*, 9(1), 2010.